

Full-Stack Technical Assessment

Software Engineer

This assessment evaluates advanced architectural thinking, coding proficiency, system design ability, API craftsmanship, frontend expertise, and judgment in building scalable and maintainable enterprise applications using ASP.NET Core, SQL Server, and Angular.

1. Overview

You are required to develop a production-grade File Storage Service consisting of:

- A backend service using ASP.NET Core 8 (Clean Architecture)
- SQL Server for metadata storage
- Local filesystem for file storage
- A frontend client using Angular 17
- JWT authentication & authorization
- Docker-compose environment for full solution

This assignment is designed for software engineers.

It intentionally includes architectural decisions, best practice implementations, and system-level thinking beyond basic CRUD engineering.

2. Requirements

2.1 Backend – ASP.NET Core 8

Your backend must implement:

- Clean Architecture (API → Application → Domain → Infrastructure)
- SQL Server using EF Core 8 code-first migrations
- Local filesystem storage only
- Streaming uploads (no in-memory buffering)
- Streaming downloads (support for HTTP Range header)
- JWT authentication with roles: user, admin
- File metadata including:
 - Original name
 - Stored key
 - Size (bytes)
 - Content type
 - Checksum (SHA-256)
 - Tags
 - Created/Deleted timestamps
 - Version number (optional)
- Soft delete + Hard delete
- Structured logging + Correlation ID

- Configuration-based upload constraints
- Error handling using RFC 7807 ProblemDetails
- Health Checks:
 - Database reachability
 - Filesystem read/write permission check

2.2 Frontend – Angular 17

The frontend must be built using Angular 17 (standalone components) and must include:

- Login screen generating a mock JWT (no real auth backend needed)
- File upload UI:
 - Drag & drop
 - Progress indication
 - Retry handling
- File listing page:
 - Pagination
 - Filters (name, tag, content type, date range)
- File preview page:
 - Inline preview for images and PDF
- File download
- Soft delete action
- Hard delete action (admin only)
- Toast notifications for operations
- Clean code architecture:
 - core/
 - shared/
 - storage/
- One end-to-end test validating upload → list → download

2.3 Non-Functional Requirements

You must demonstrate engineering practices:

- Secure filename sanitization
- Prevent directory traversal attacks
- Atomic file writes (temp → commit)
- Streaming large files (100 MB test)

- Separation of domains and boundaries
- Dependency injection best practices
- Clear error models and validation
- Logging with correlation IDs
- Respect configuration for max upload size
- Document all assumptions and decisions

3. Solution Architecture

Your architecture is expected to show clarity and modularity. 3.1

Backend Layers:

- API Layer — controllers, filters, problem handling
- Application Layer — business logic, services, validation
- Domain Layer — entities, aggregates, enums, rules
- Infrastructure Layer — EF Core, repositories, local storage implementation

3.2 File System Layout (Local Storage):

```
_storage/  
yyyy/  
MM/  
dd/  
/  
content.bin  
metadata.json (optional)
```

3.3 Database Structure:

Table: StoredObjects

- Id (GUID)
- Key (string)
- OriginalName
- SizeBytes
- ContentType
- Checksum
- Tags
- CreatedAtUtc
- DeletedAtUtc (nullable)
- Version

- CreatedByUserId

3.4 API Endpoints:

POST /api/files

GET /api/files

GET /api/files/{id}/download

GET /api/files/{id}/preview

DELETE /api/files/{id}

DELETE /api/files/{id}/hard

GET /health/live

GET /health/ready

4. Evaluation Criteria (100 points)

Your submission will be evaluated on the following weighted criteria:

- Architecture Quality (20 points)
- Backend Implementation Correctness (20 points)
- Security, Validation & Error Handling (15 points)
- Frontend Architecture & UI/UX (15 points)
- Coding Style & Maintainability (10 points)
- Streaming Performance & FS Safety (10 points)
- Testing Coverage (Unit + Integration + E2E) (5 points)
- Documentation & README Quality (5 points)

Bonus Points (Optional):

- Implement resumable uploads
- Implement audit log
- Implement ETag-based caching

5. Submission Instructions

Please submit:

1. A GitHub/GitLab/Bitbucket repository containing:

- Backend solution
- Frontend solution
- docker-compose.yml

- SQL migrations
- README.md

2. README must include:

- Setup steps
- Architecture overview
- Design decisions
- Known limitations
- Future enhancements

Time Expectation:

This assignment typically requires 8–12 hours.

Quality is more important than speed.

6. Final Notes

This assessment simulates real-world system design and implementation expected from a Software Engineer.

We are evaluating:

- How you think about architecture
- How you structure code
- How you protect and secure systems
- How you design APIs and modules
- How you handle complexity
- How you communicate technical decisions

Good luck — and build it as if it were going into production.